



Improving Checkpointing I/O Performances of the Gysela HPC Application

Alexandre Lou-Poueyou, Francieli Boito, Méline Trochon, Luan Teylo

**TECHNICAL
REPORT**

N° XXXX

August 2026

Project-Teams TADaaM



Improving Checkpointing I/O Performances of the Gysela HPC Application

Alexandre Lou-Poueyou*, Francieli Boito[†], Méline Trochon[‡],

Luan Teylo[§]

Project-Teams TADaaM

Technical Report n° XXXX — August 2026 — 10 pages

Abstract: In modern High-Performance Computing (HPC) systems, Input/Output (I/O) performance represents a critical bottleneck for large-scale scientific simulations. As computational power scales towards exascale, storage bandwidth has not kept pace, creating a significant compute-to-I/O performance gap. Saving application states through periodic checkpointing is essential for fault tolerance, yet synchronous checkpoint writing halts computation and wastes expensive CPU/GPU hours.

This work focuses on optimizing the checkpointing performance of Gysela, a 5D gyrokinetic simulation code developed by the CEA for nuclear fusion research. Gysela generates massive datasets and relies on periodic checkpointing for execution restartability. Using `gys_io`, a lightweight mini-application replicating Gysela's parallel 5D I/O patterns, we systematically evaluate three parallel I/O strategies—Single Shared File (SSF), Subfilng, and File Per Process (FPP)—alongside Parallel File System (PFS) striping configurations on clusters.

Key-words: HPC, Parallel I/O, Performance Tuning, Parallel File Systems

* Computer Science Master's Student at University of Bordeaux – Intern at TADaaM team, Inria Bordeaux

[†] Associate Professor at University of Bordeaux & Permanent Researcher in TADaaM team, Inria Bordeaux

[‡] PhD Student in TADaaM team, Inria Bordeaux

[§] Permanent Researcher in TADaaM team, Inria Bordeaux

RESEARCH CENTRE
Centre Inria de l'Université de Bordeaux

200 avenue de la Vieille Tour
33405 Talence Cedex

Contents

1	Introduction	4
2	Institutional & Project Context	5
3	Background & High-Level Write Strategies	6
3.1	Parallel File Systems and Data Striping	6
3.2	Parallel Write Strategies	7
3.2.1	Single Shared File (SSF)	7
3.2.2	Subfiling	7
3.2.3	File Per Process (FPP)	8
3.3	Software Stack and Tooling	8
4	Experimental Setup	8
4.1	Hardware and Storage Infrastructure	8
4.2	Benchmark Workload & Evaluation Parameters	9

1 Introduction

In modern High-Performance Computing (HPC) systems, Input/Output (I/O) throughput has become as critical to overall efficiency as computational capability itself. As supercomputing architectures advance toward and beyond the exascale threshold, computational throughput has increased exponentially. However, persistent storage bandwidth has lagged behind, widening the compute-to-I/O performance gap. For instance, Figure 1 shows how the ratio between I/O bandwidth and compute power has evolved over the past two decades for the machines listed in the TOP500 [2] and IO500 [1] rankings. Since 2000, the I/O bandwidth available per unit of compute has dropped by roughly $50\times$, from 312 GB/s per PFlop/s on ASCI Red down to 6.0 GB/s per PFlop/s on Frontier, with only a modest recovery on the more recent Aurora system.

Therefore, ~~W~~ while modern processors achieve multi-petaflop or exaflop performance, parallel file systems typically deliver only hundreds of gigabytes to a few terabytes per second in aggregate write bandwidth. This growing imbalance creates a severe bottleneck: high-performance computing tasks spend a non-negligible fraction of their allocated runtime waiting for data transfers to complete. ~~The compute-to-I/O ratio (measured in GB/s per PetaFlop) has degraded by more than an order of magnitude over the past two decades.~~

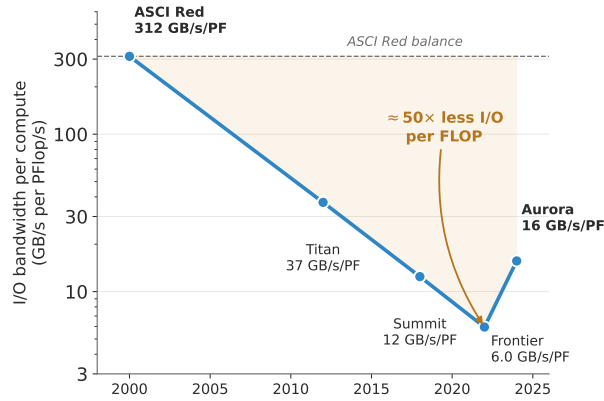


Figure 1: Compute-to-I/O performance gap in Top500 supercomputers (2000–2025). The ratio of storage bandwidth per unit of compute has degraded by more than an order of magnitude.

Large-scale scientific simulations generate massive volumes of data requiring efficient parallel management. On shared Parallel File Systems (PFS), thousands of concurrent processes must co-ordinate access to shared storage arrays while contending for network link bandwidth and storage controller resources. ~~An prominent~~ example of an application with strong I/O pressure is Gysela, a 5D gyrokinetic simulation code developed by ~~the~~ CEA (*Commissariat à l'Énergie Atomique et aux Énergies Alternatives*) for nuclear fusion research. Gysela models plasma turbulence in magnetic confinement devices (tokamaks) by solving the 5D Vlasov-Poisson system ~~[TODO: add citation]~~. ~~across three spatial coordinates (minor radius r , poloidal angle θ , toroidal angle φ) and two velocity-space coordinates (parallel velocity $v_{\parallel}$, magnetic moment μ). Understanding and mitigating plasma turbulence is crucial for maximizing thermal confinement in next-generation fusion reactors such as ITER.~~

Because ~~production~~ Gysela ~~simulations execute~~ executes across hundreds to thousands of computing nodes over extended periods (days to weeks), ~~they rely~~ it relies on periodic checkpointing for fault tolerance and job restartability. The checkpointing procedure is synchronous: all parallel processes pause numerical integration, aggregate their local state vectors, and flush the

5D distribution function to persistent storage. A single checkpoint operation can generate terabytes of data, putting strong pressure on the storage system and interfering with the other applications that share it. ~~During~~ Moreover, during this phase, compute resources remain idle ~~Moreover~~, and in shared production environments, ~~checkpoint~~ the write times exhibit high variability—sometimes exceeding median write durations by a factor of 3 ~~do you have a citation for that?~~—due to concurrent I/O contention from uncoordinated tenant jobs.

Optimizing ~~checkpoint~~ I/O performance requires a coordinated approach across two complementary layers: the ~~Parallel File System (PFS) Configuration~~ PFS configuration, which ~~involve~~ involves adjusting low-level layout parameters, such as striping count (number of storage targets) and stripe size, and ~~High-Level I/O Strategies depends which~~ high-level I/O strategies, which determine how application data structures are mapped, aggregated, and formatted before ~~being~~ being dispatched to storage (e.g., Single Shared File, Subfiling, or File Per Process). In this work, we explore both layers: we conduct a tuning study over several parameters that affect the I/O behaviour of Gysela’s checkpoint procedure. Our objective is to determine which configuration maximizes I/O performance, and to characterize how each parameter contributes to it.

To enable rapid, reproducible experimentation without incurring the massive computational cost of full plasma physics simulations, we employ `gys_io`, a dedicated mini-application that faithfully reproduces Gysela’s 5D checkpoint I/O access pattern ~~[TODO: add citation]~~. ~~In this report, we evaluate the impact of PFS parameters and high-level write strategies, analyze scaling behaviors on the Plafim benchmark platform, and formulate actionable deployment recommendations for Gysela production runs.~~ The remainder of this report is organized as follows. Section 2 presents the institutional and project context of this work. Section 3 reviews how parallel file systems distribute data across storage targets and describes the three parallel write strategies we evaluate, together with the software stack used to run the experiments. Section 4 details the experimental setup: the clusters and their storage systems, the benchmark workload, and the range of parameters explored. ~~[TODO: extend this roadmap once the results and conclusion sections exist]~~

2 Institutional & Project Context

This work was conducted as part of a Master’s research internship at Inria (the French National Institute for Research in Digital Science and Technology) within the TADaaM research team at the Inria Centre at the University of Bordeaux. TADaaM (*Topology-Aware system-scale Data management for high-performance computing*) specializes in designing system software and data management layers that bridge the gap between application-level requirements and complex hardware topologies.

This research contributes directly to the NumPex program, a major French national initiative designed to build the software ecosystem for exascale supercomputing. Specifically, the study falls under the Exa-DOST work package, which focuses on developing advanced I/O optimization strategies, dynamic data management middlewares, and workflow scheduling tools for extreme-scale applications.

The target application, Gysela, is maintained and advanced by researchers at CEA. Collaborative efforts between Inria TADaaM and CEA aim to prepare Gysela’s I/O substrate for upcoming European exascale supercomputers (such as Alice Recoque in France).

- **Inria and the Inria Centre at the University of Bordeaux.**

- Inria: French national research institute for digital science and technology, organised in research centres, each hosting project-teams that are joint with the university.

- The Inria Centre at the University of Bordeaux: **[TODO: number of teams and staff, main research axes, location on the Talence campus]**.
- Local ecosystem relevant to this work: HPC culture of the lab
- **The host team.**
 - Team TADAAM
 - Position of this internship inside the team’s roadmap: storage performance characterisation, I/O benchmarking, and the development of the IOPS framework.
 - Supervision: Francieli, Meline...
 - People involved in the work. CEA people...
- **Scientific and technical environment.**
 - **PlaFRIM**: the experimental platform used for all the measurements. **[TODO: describe: number of nodes, the bora partition used here (CPU model, cores per node, memory, interconnect), SLURM as the scheduler]**.
 - **Storage**: BeeGFS
 - **Software stack**: SLURM, OpenMPI **[TODO: 4.0.1 was used, confirm]**, Python 3.10 or later, IOPS,
- **Organisation of the work.**
 - Timeline of the internship: When it start, when it finish. The GYSWEEK.
 - Working method: weekly meetings, Git/GitLab repositories, notebooks and reproducible scripts.
 - Deliverables produced: the study repository, the contributions to IOPS , this report, and the intended paper. The new contract.

3 Background & High-Level Write Strategies

3.1 Parallel File Systems and Data Striping

Traditional file systems rely on single storage controllers, which creates a severe bottleneck when thousands of parallel MPI ranks attempt simultaneous file access. Parallel File Systems (PFS)—such as Lustre, BeeGFS, and many others—bypass this bottleneck by partitioning files across multiple independent storage devices via data striping.

In such systems, storage capacity is divided into logical entities called Object Storage Targets (OSTs). An OST represents a dedicated physical storage unit (e.g., a RAID array or NVMe volume) managed by an Object Storage Server (OSS). Metadata servers (MDS) maintain directory structures and file layouts, while OSTs handle actual data payload transfers.

Data striping partitions a logical file stream into contiguous blocks of size S_{stripe} (Stripe Size). These blocks are distributed in a round-robin sequence across N_{OST} targets (Stripe Count or OST Count). For example, with $N_{\text{OST}} = 4$ and $S_{\text{stripe}} = 1$ MB, the first megabyte is written to OST 0, the second to OST 1, and so on. Striping allows applications to aggregate the physical bandwidth of multiple storage devices, ideally achieving an N_{OST} -fold increase in overall write throughput.

3.2 Parallel Write Strategies

The layout of output files significantly impacts storage efficiency. In this study, we analyze three primary high-level I/O strategies, implemented via the Parallel Data Interface (PDI) middleware:

3.2.1 Single Shared File (SSF)

In the Single Shared File approach, all MPI processes concurrently write their local data slices into a single unified file on the PFS (Figure 2). Standard libraries (such as HDF5 or MPI-IO) employ two-phase I/O to aggregate non-contiguous memory layouts into large, contiguous file offsets before issuing collective storage calls. While SSF simplifies dataset management by producing a single output file, it can suffer from locking contention on storage targets when rank counts scale up.

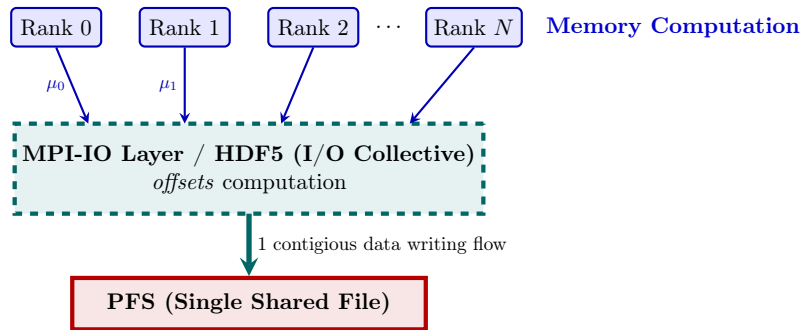


Figure 2: Single Shared File (SSF) strategy overview.

3.2.2 Subfiling

Subfiling acts as an intermediate approach between SSF and individual file writes. MPI processes are partitioned into distinct sub-groups. Within each group, an assigned process—referred to as an I/O Concentrator (IOC) or aggregator—collects data from its peer ranks and writes it to a dedicated subfile (Figure 3).

This reduces file system lock contention compared to SSF, while avoiding the metadata overload associated with generating thousands of individual files. The total number of generated subfiles (N_{subfiles}) is a tunable parameter.

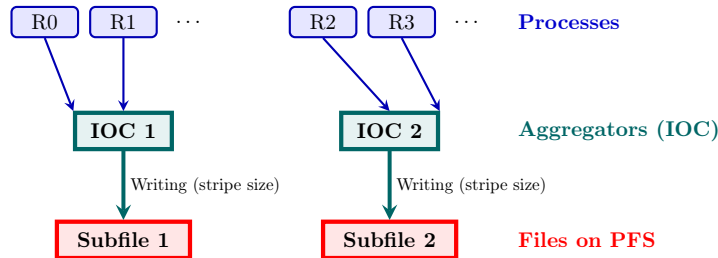


Figure 3: Subfiling strategy overview with two I/O Concentrators (IOCs).

3.2.3 File Per Process (FPP)

In the File Per Process approach, every MPI process creates and writes its own separate file independently (Figure 3.2.3). Because there is no inter-process communication or offset synchronization required, local write throughput can be very high. However, FPP generates N_{ranks} separate files per checkpoint, which can strain metadata servers (MDS) at scale and complicate downstream post-processing or data management.

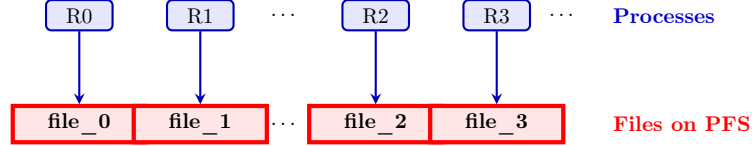


Figure 4: File Per Process (FPP) strategy overview.

3.3 Software Stack and Tooling

To evaluate these strategies, our experimental framework combines three key software components:

- **gys_io Mini-App:** Developed by CEA, gys_io is a lightweight benchmark proxy that replicates Gysela’s exact 5D domain decomposition and checkpoint write calls while skipping computational physics routines. It reads Gysela configuration files directly, enabling fast, reproducible I/O benchmarks.
- **Parallel Data Interface (PDI):** PDI decouples application code from underlying I/O libraries. By modifying external YAML configuration files, PDI allows switching between SSF, Subfiling, and FPP backends without recompiling the application binary.
- **IOPS Benchmark Automation:** IOPS automates parameter sweeps, manages SLURM batch job submission, and extracts execution metrics (such as `time_write` and aggregate write bandwidth).

4 Experimental Setup

To evaluate the performance and scalability of our parallel I/O strategies, we conducted experiments across two different High-Performance Computing (HPC) clusters: a regional cluster used for initial setup and prototyping, and a national supercomputer used for larger-scale runs.

4.1 Hardware and Storage Infrastructure

The first phase of our experiments took place on the Plafrim cluster at Inria Bordeaux. Benchmarks were deployed on the Bora compute nodes, which are equipped with dual-socket Intel Xeon Gold 6240 processors (36 physical cores per node at 2.60 GHz) and 192 GB of RAM per node. Inter-node communications relied on an Intel Omni-Path network delivering 100 Gb/s bandwidth. Storage on Plafrim is managed by a BeeGFS parallel file system offering an aggregate bandwidth capped at 10 Gbit/s and configured with up to 8 active Object Storage Targets (OSTs). This platform provided a controlled environment to validate our setup and measure baseline performance. To evaluate our strategies at a larger scale, we extended our benchmark

campaign to the Adastral supercomputer at CINES. Specifically, tests were run on the GENOA CPU partition, which features dual-socket AMD EPYC 9654 processors with 192 physical cores per node and high memory throughput. Nodes are connected via an HPE Slingshot-11 high-speed network. Storage on Adastral is provided by a production-grade Lustre parallel file system, which allows fine-grained data striping across up to 40 active OSTs.

4.2 Benchmark Workload & Evaluation Parameters

The experiments considered several 5D grid configurations representative of different Gysela workloads. The evaluated resolutions and their corresponding single-checkpoint memory footprints are summarized below:

$$\begin{aligned} C_1 : N_r \times N_\theta \times N_\varphi \times N_{v_\parallel} \times N_\mu &= 128 \times 128 \times 64 \times 64 \times 64 \\ C_2 : N_r \times N_\theta \times N_\varphi \times N_{v_\parallel} \times N_\mu &= 256 \times 512 \times 128 \times 128 \times 64 \\ C_3 : N_r \times N_\theta \times N_\varphi \times N_{v_\parallel} \times N_\mu &= 512 \times 1024 \times 64 \times 128 \times 64 \end{aligned} \tag{1}$$

These configurations result in single-checkpoint memory footprints ranging from approximately 64 GB to ... GB. Each configuration was evaluated using the different process topologies considered in the experiments, allowing their impact on I/O performance to be assessed independently of the problem size.

Benchmarks were submitted in exclusive node allocation mode with 8 tasks per node, across process scales ranging from 16 MPI ranks (2 nodes) up to MPI ranks 2048 (256 nodes). To account for background network and storage noise, each experimental point represents the average of 5 independent executions.

References

- [1] IO500 Foundation. The IO500 list. <https://io500.org/>, 2026. Accessed: August 18, 2026.
- [2] TOP500.org. The TOP500 list. <https://www.top500.org/>, 2026. Accessed: August 18, 2026.

The Inria logo is a red, stylized script wordmark that reads "Inria". It is positioned inside a white rectangular box with rounded corners, which is set against a light gray background.

RESEARCH CENTRE
Centre Inria de l'Université de Bordeaux

200 avenue de la Vieille Tour
33405 Talence Cedex

Publisher
Inria
Domaine de Voluceau - Rocquencourt
BP 105 - 78153 Le Chesnay Cedex
inria.fr

ISSN 0249-0803